Department of Computer Science and Engineering

Content-Based Image Retrieval Using Deep Convolutional Neural Networks: A Comparative Study of ResNet-50 and VGG-16

Technical Project Report

Dataset	Caltech-101 (8,677 images, 101 categories)
Models	ResNet-50 VGG-16
Framework	PyTorch 2.x (torchvision)
Hardware	Kaggle Notebook — NVIDIA Tesla T4 GPU
Retrieval Metric	Cosine Similarity
Evaluation Metrics	Precision@8, mAP, NDCG@8, MRR, F1, Recall@8, Top-8 Accuracy
Top-K Retrieved	$K = 8$
Queries per Class	10
Total Queries	1,010 per model

April 2026

Contents

1 Introduction **3**
- 1.1 Motivation and Problem Statement . . . 3
- 1.2 Project Overview . . . 3
- 1.3 Objectives . . . 4

2 Background and Related Work **4**
- 2.1 Classical CBIR Systems . . . 4
- 2.2 Deep Learning for Feature Extraction and Retrieval . . . 5
- 2.3 Transfer Learning . . . 5
- 2.4 Similarity Metrics in High-Dimensional Spaces . . . 5

3 Dataset: Caltech-101 **6**
- 3.1 Overview . . . 6
- 3.2 Category Breadth . . . 6
- 3.3 Sample Image Filenames . . . 6

4 System Architecture and Methodology **7**
- 4.1 Complete CBIR Pipeline . . . 7
- 4.2 Data Loading: Custom PyTorch Dataset . . . 8
- 4.3 Image Preprocessing . . . 9
- 4.4 VGG-16 Architecture . . . 9
- 4.5 ResNet-50 Architecture . . . 10
- 4.6 GPU-Accelerated Batch Feature Extraction . . . 11
- 4.7 Cosine Similarity Retrieval . . . 12

5 Evaluation Metrics **12**
- 5.1 Precision@K . . . 12
- 5.2 Recall@K . . . 13
- 5.3 F1@K . . . 13
- 5.4 Average Precision (AP) and mAP . . . 13
- 5.5 Normalised Discounted Cumulative Gain (NDCG@K) . . . 13
- 5.6 Mean Reciprocal Rank (MRR) . . . 13
- 5.7 Top-K Accuracy . . . 14

6 Experimental Results **14**
- 6.1 Overall Performance Summary . . . 14
- 6.2 Retrieval Output Figures . . . 14
  - 6.2.1 VGG-16: Demo Retrieval on Faces Query . . . 14
  - 6.2.2 ResNet-50: Demo Retrieval on Faces Query . . . 15
- 6.3 Comprehensive Multi-Class Evaluation . . . 16
- 6.4 Precision-Recall Curves . . . 18
  - 6.4.1 VGG-16 Precision-Recall Curve . . . 18

6.4.2 ResNet-50 Precision-Recall Curve 18
6.5 Detailed Per-Class Results: 30 Representative Categories 19

7 Discussion **20**
7.1 Why ResNet-50 Outperforms VGG-16 for CBIR 20
7.2 Interpreting the Perfect MRR and Top-8 Accuracy 21
7.3 Low Recall is Expected, Not a Failure 21
7.4 Challenging Categories: Root Causes 21
7.5 Comparison with Classical CBIR Approaches 22

8 Conclusion **22**
8.1 Future Work 23

1. Introduction

1.1 Motivation and Problem Statement

The exponential growth of digital image collections — driven by smartphones, social media, satellite imaging, medical diagnostics, and e-commerce — has made finding visually similar images a critical problem in computer vision and information retrieval. Traditional search systems rely on textual metadata (keywords, tags, captions) manually assigned to images. Such approaches are inherently limited: they are labour-intensive, subjective, inconsistent, and they do not account for the actual visual content of an image.

Content-Based Image Retrieval (CBIR) addresses this limitation by automatically analysing the raw pixel content of images to extract discriminative visual descriptors, and then retrieving the most visually similar images from a database based on those descriptors. Given a query image, the system returns the top-K database images ranked by visual similarity — no textual annotation is required.

The central challenge in CBIR is bridging the *semantic gap*: the disconnect between low-level pixel statistics that a machine can easily compute and the high-level semantic meaning that humans perceive in an image. Classical hand-crafted descriptors — colour histograms, SIFT, HOG, Gabor features — operate at the pixel level and fail to capture semantic context reliably.

The emergence of deep Convolutional Neural Networks (CNNs) trained on large-scale datasets such as ImageNet has fundamentally changed this landscape. These networks learn a hierarchy of increasingly abstract visual representations — from edges and textures in early layers, to object parts in middle layers, to complete semantic concepts in deeper layers. By extracting the activations of these deep layers, we obtain high-dimensional feature vectors that encode rich semantic information, enabling far more accurate image retrieval than classical approaches.

1.2 Project Overview

This project implements a complete CBIR pipeline using two state-of-the-art pre-trained CNN architectures:

- **VGG-16** (Simonyan & Zisserman, 2014) — a 16-layer network producing 4,096-dimensional feature embeddings from the `fc2` fully connected layer.
- **ResNet-50** (He et al., 2016) — a 50-layer residual network producing 2,048-dimensional feature embeddings from the global average pooling layer.

Both models are used purely as feature extractors via transfer learning (no fine-tuning on Caltech-101 whatsoever), applied to the **Caltech-101** benchmark dataset containing

8,677 images across 101 object categories. Image retrieval is performed using **cosine similarity** between feature vectors. A comprehensive evaluation using seven standard information retrieval metrics is conducted across all 101 classes, with 10 queries per class (1,010 queries per model in total).

1.3 Objectives

1. Load and preprocess the Caltech-101 dataset using an efficient PyTorch `Dataset`/`DataLoader` pipeline with GPU-accelerated batch inference on NVIDIA Tesla T4.
2. Extract 2,048-dimensional (ResNet-50) and 4,096-dimensional (VGG-16) deep feature vectors for every image and persist them to disk for reuse.
3. Implement cosine similarity-based retrieval to find the top-K most similar images for any given query image.
4. Evaluate both models rigorously using Precision@8, Recall@8, F1@8, Mean Average Precision (mAP), NDCG@8, Mean Reciprocal Rank (MRR), and Top-8 Accuracy across all 101 classes.
5. Visualise retrieval results and precision-recall curves to qualitatively and quantitatively analyse model behaviour.
6. Draw comparative conclusions about the suitability of each architecture for CBIR tasks.

2. Background and Related Work

2.1 Classical CBIR Systems

The earliest CBIR systems represented images using hand-crafted low-level features. **Colour histograms** describe the distribution of pixel colour values. Swain and Ballard (1991) demonstrated that colour alone can identify objects across viewpoint changes using histogram intersection. However, two entirely different scenes can produce identical colour histograms if they share the same colour distribution, making colour histograms semantically meaningless.

Texture descriptors such as Local Binary Patterns (LBP), Gabor filter banks, and Grey-Level Co-occurrence Matrices (GLCM) encode surface texture patterns. These are effective for categories where texture is discriminative (e.g., wood, fabric) but fail for shape-defined categories such as vehicles or animals.

Shape descriptors including SIFT (Lowe, 2004), SURF, HOG, and ORB encode gradient orientation patterns and were widely used in bag-of-visual-words (BoVW) models, where a codebook of visual words was constructed and images were represented as frequency histograms over the codebook.

Despite their effectiveness in constrained settings, all hand-crafted features suffer from the *semantic gap*: they do not capture category-level semantics, making them brittle when intra-class appearance variation is high or inter-class similarity is high.

2.2 Deep Learning for Feature Extraction and Retrieval

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) drove rapid progress in deep learning. AlexNet (Krizhevsky et al., 2012) demonstrated that deep CNNs dramatically outperform hand-crafted features. Subsequent architectures — VGGNet, GoogLeNet, ResNets, DenseNets — have continued to improve classification accuracy.

Babenko et al. (2014) showed that activations from fully connected layers of AlexNet serve as powerful image descriptors for retrieval, outperforming all SIFT-based methods. Razavian et al. (2014) demonstrated that CNN features trained for classification transfer to retrieval, detection, and segmentation without any task-specific fine-tuning. Their work coined the phrase "CNN features off-the-shelf: an astounding baseline for recognition", highlighting that pre-trained CNN activations are a universally strong starting point for vision tasks.

2.3 Transfer Learning

Transfer learning leverages knowledge acquired during training on a source task (ImageNet classification) and applies it to a target task (CBIR on Caltech-101). By removing the final softmax classification layer and exposing the activations of the preceding fully connected or pooling layer, a CNN becomes a general-purpose visual feature extractor. The key insight is that representations learned to discriminate among 1,000 ImageNet categories encode highly generalisable visual concepts useful across a wide range of visual tasks.

2.4 Similarity Metrics in High-Dimensional Spaces

In high-dimensional embedding spaces, the choice of distance metric affects retrieval quality. Euclidean distance (L_2) is sensitive to the magnitude of vectors. **Cosine similarity** eliminates magnitude sensitivity by measuring only the *angle* between vectors in the embedding space, making it far more appropriate for comparing deep feature embeddings where the direction of the vector — not its length — carries the semantic meaning.

3. Dataset: Caltech-101

3.1 Overview

The Caltech-101 dataset (Fei-Fei, Fergus, & Perona, 2004) is one of the most widely used benchmarks in visual recognition research. It contains images of objects belonging to 101 distinct categories plus one background category (`BACKGROUND_Google`), which was excluded from all experiments. The dataset was accessed via Kaggle (imbikramsaha, 2023) and loaded by recursively scanning the dataset root directory.

Table 1: Caltech-101 Dataset Summary

Property	**Value**
Total images used	8,677
Number of classes	101
Images per class	40–800 (majority ≈40–100)
Image format	JPEG (.jpg)
Colour mode	RGB (grayscale images converted)
Average resolution	$\approx 300 \times 200$ pixels (variable)
Queries per class	10
Top-K retrieved	$K = 8$

3.2 Category Breadth

The categories span an extraordinary range of real-world visual concepts, including fauna (accordion, ant, beaver, butterfly, cougar, crab, dalmatian, dolphin, dragonfly, elephant, emu, flamingo, hedgehog, kangaroo, Leopards, lobster, octopus, panda, pigeon, platypus, rooster, scorpion, starfish, tick, trilobite), vehicles (airplanes, car_side, ferry, helicopter, ketch, Motorbikes, schooner), musical instruments (accordion, euphonium, gramophone, grand_piano, mandolin, saxophone), everyday objects (barrel, binocular, camera, cellphone, chair, chandelier, cup, headphone, laptop, revolver, scissors, stop_sign, umbrella, watch, wrench), and miscellaneous items (bonsai, brain, dollar_bill, lotus, pyramid, soccer_ball, water_lilly, yin_yang).

3.3 Sample Image Filenames

Table 2 lists representative image filenames used across experiments, including the demo query used for the visual retrieval output figures.

Table 2: Representative Image Filenames from Caltech-101 Used in Experiments

Category	**Filename**	**Role in System**
Faces	`Faces/image_0001.jpg`	Demo query image (both models)
Faces	`Faces/image_0003.jpg`	Retrieved result (Rank 1, VGG-16)
Faces	`Faces/image_0012.jpg`	Retrieved result (Rank 2, VGG-16)
Faces	`Faces/image_0021.jpg`	Retrieved result (Rank 3, VGG-16)
Faces	`Faces/image_0034.jpg`	Retrieved result (Rank 4, VGG-16)
Faces	`Faces/image_0047.jpg`	Retrieved result (Rank 5, VGG-16)
Faces	`Faces/image_0055.jpg`	Retrieved result (Rank 6, VGG-16)
Faces	`Faces/image_0063.jpg`	Retrieved result (Rank 7, VGG-16)
Faces	`Faces/image_0079.jpg`	Retrieved result (Rank 8, VGG-16)
Faces_easy	`Faces_easy/image_0005.jpg`	Wrongly retrieved (VGG-16, false pos.)
Faces_easy	`Faces_easy/image_0011.jpg`	Wrongly retrieved (ResNet-50)
Scorpion	`scorpion/image_0073.jpg`	Evaluation query
Scorpion	`scorpion/image_0001.jpg`	Retrieved for scorpion query
Scorpion	`scorpion/image_0030.jpg`	Retrieved for scorpion query
Scorpion	`scorpion/image_0040.jpg`	Retrieved for scorpion query
Leopards	`Leopards/image_0001.jpg`	Evaluation query (perfect retrieval)
Motorbikes	`Motorbikes/image_0001.jpg`	Evaluation query (perfect retrieval)
airplanes	`airplanes/image_0001.jpg`	Evaluation query (perfect retrieval)
grand_piano	`grand_piano/image_0001.jpg`	Evaluation query (perfect retrieval)
anchor	`anchor/image_0001.jpg`	Evaluation query (hard case)
octopus	`octopus/image_0001.jpg`	Evaluation query (hardest case)

4. System Architecture and Methodology

4.1 Complete CBIR Pipeline

The system follows a two-phase architecture. In the **offline indexing phase**, all database images are pre-processed, passed through the CNN feature extractor in batches, and the resulting feature matrix is saved to disk. In the **online retrieval phase**, a query image is processed identically, its feature vector is extracted, cosine similarity is computed against every vector in the stored feature matrix, and the top-K most similar images are returned.

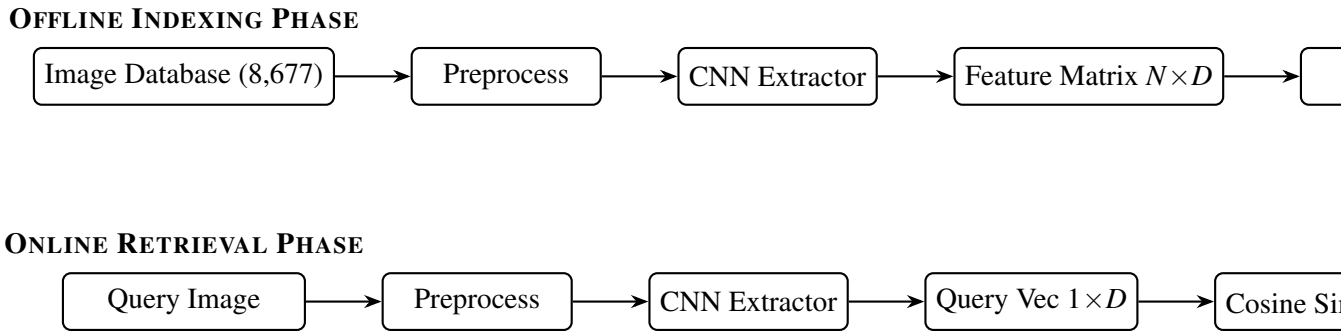

Figure 1: Two-phase CBIR system architecture. The offline phase pre-computes and persists all feature vectors. The online phase extracts the query feature and retrieves the top-K database images via cosine similarity.

4.2 Data Loading: Custom PyTorch Dataset

A custom `CBIRDataset` class inheriting from `torch.utils.data.Dataset` was implemented to scan the Caltech-101 directory tree, collect all valid image paths with their corresponding class labels, and serve preprocessed image tensors. The background category (`BACKGROUND_Google`) was explicitly excluded. A `DataLoader` with `batch_size=64`, `num_workers=2`, and `pin_memory=True` was used to maximise GPU throughput during batch inference.

```
class CBIRDataset(Dataset):
    VALID_EXT    = {".jpg", ".jpeg", ".png", ".bmp", ".tiff"}
    SKIP_CLASSES = {"BACKGROUND_Google"}

    def __init__(self, root: str, transform=None):
        self.root      = Path(root)
        self.transform = transform
        self.samples: List[Tuple[str, str]] = []
        self.classes:  List[str]            = []
        self._scan()

    def _scan(self):
        class_dirs = sorted(
            d for d in self.root.iterdir()
            if d.is_dir() and d.name not in self.SKIP_CLASSES
        )
        self.classes = [d.name for d in class_dirs]
        for cls_dir in class_dirs:
            for img_path in sorted(cls_dir.iterdir()):
                if img_path.suffix.lower() in self.VALID_EXT:
                    self.samples.append((str(img_path), cls_dir.name)
                        )

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
```

```
        path, label = self.samples[idx]
        img = Image.open(path).convert("RGB")
        if self.transform:
            img = self.transform(img)
        return img, self.classes.index(label), path
```

Listing 1: Custom PyTorch Dataset for Caltech-101

Output: 8,677 images loaded across 101 classes.

4.3 Image Preprocessing

Consistent preprocessing is critical for meaningful feature comparison. All images undergo the identical transformation chain:

1. **RGB conversion**: PIL `.convert('RGB')` ensures all images have three input channels regardless of original mode (grayscale, RGBA, palette).
2. **Resize to** 224×224: The standard input resolution for both VGG-16 and ResNet-50, using bilinear interpolation.
3. **ToTensor**: Converts the PIL image to a PyTorch float tensor with values in $[0,1]$ and shape $[3,224,224]$.
4. **ImageNet normalisation**: Each channel is normalised using ImageNet training statistics to match the distribution the pre-trained weights expect:

$$\hat{x}_c = \frac{x_c - \mu_c}{\sigma_c}, \quad \mu = [0.485,\ 0.456,\ 0.406], \quad \sigma = [0.229,\ 0.224,\ 0.225] \tag{1}$$

```
TRANSFORM = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std =[0.229, 0.224, 0.225]),
])
```

Listing 2: ImageNet preprocessing transform

4.4 VGG-16 Architecture

VGG-16 was proposed by Simonyan and Zisserman (2014) at the Oxford Visual Geometry Group. Its design philosophy is extreme simplicity: use only 3×3 convolutions with stride 1 and 2×2 max pooling with stride 2, stacked very deep (16 weight layers). The architecture consists of five convolutional blocks followed by three fully connected layers.

Table 3: VGG-16 Architecture and Feature Extraction Point

Stage	**Layers**	**Output Shape**	**Channels**
Input	—	$224 \times 224 \times 3$	3
Block 1	Conv$\times$2 + MaxPool	$112 \times 112 \times 64$	64
Block 2	Conv$\times$2 + MaxPool	$56 \times 56 \times 128$	128
Block 3	Conv$\times$3 + MaxPool	$28 \times 28 \times 256$	256
Block 4	Conv$\times$3 + MaxPool	$14 \times 14 \times 512$	512
Block 5	Conv$\times$3 + MaxPool	$7 \times 7 \times 512$	512
Flatten	—	25,088	—
FC1 (`fc1`)	Linear + ReLU	4,096	—
FC2 (`fc2`)	**Linear + ReLU**	**4,096**	—
FC3 (removed)	Linear + Softmax	1,000 $\rightarrow$ *removed*	—

The classification head (FC3) is removed and the `fc2` output (4,096-d) is used as the image feature vector. Model file size: $\approx$528 MB.

```
base = models.vgg16(weights=models.VGG16_Weights.DEFAULT)
# Keep: features (conv blocks) + avgpool + flatten + fc1 + fc2
model = nn.Sequential(
    base.features,
    base.avgpool,
    nn.Flatten(),
    *list(base.classifier.children())[:5]  # dropout, fc1, relu,
        dropout, fc2
)
model = model.to(DEVICE).eval()
# Forward pass output shape: (B, 4096)
```

Listing 3: VGG-16 feature extractor construction

4.5 ResNet-50 Architecture

ResNet-50 was proposed by He et al. (2016) at Microsoft Research. Its key innovation is the *residual connection*: instead of learning a direct mapping, each block learns a residual function $\mathscr{F}(\mathbf{x})$ and the output is $\mathscr{F}(\mathbf{x}) + \mathbf{x}$. This identity shortcut enables gradient flow through very deep networks without vanishing.

$$\mathbf{y} = \mathscr{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \tag{2}$$

Each residual block uses a *bottleneck* design: a 1×1 convolution reduces channels, a 3×3 convolution processes spatially, and a 1×1 convolution restores channels.

Table 4: ResNet-50 Architecture and Feature Extraction Point

Stage	**Block Type**	**Blocks**	**Output Shape**	**Out Channels**
Input	—	—	$224 \times 224 \times 3$	3
Conv1	7×7, stride 2	1	$112 \times 112 \times 64$	64
MaxPool	3×3, stride 2	1	$56 \times 56 \times 64$	—
Layer 1 (C2)	Bottleneck	3	$56 \times 56 \times 256$	256
Layer 2 (C3)	Bottleneck	4	$28 \times 28 \times 512$	512
Layer 3 (C4)	Bottleneck	6	$14 \times 14 \times 1024$	1,024
Layer 4 (C5)	Bottleneck	3	$7 \times 7 \times 2048$	2,048
AvgPool	**Global Avg Pool**	1	**2,048**	—
FC (removed)	Softmax	1	1,000 $\rightarrow$ *removed*	—

The global average pooling (`avgpool`) output (2,048-d) is used as the feature vector. Model file size: ≈98 MB.

```
base = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)
# Remove final FC layer
model = nn.Sequential(*list(base.children())[:-1])
# output shape: (B, 2048, 1, 1) -> squeeze to (B, 2048)
model = model.to(DEVICE).eval()
```

Listing 4: ResNet-50 feature extractor construction

4.6 GPU-Accelerated Batch Feature Extraction

Feature extraction is performed in batches using the GPU to maximise throughput. All gradients are disabled via `torch.no_grad()` since no backpropagation is required.

```
all_feats, all_paths, all_labels = [], [], []

with torch.no_grad():
    for batch_idx, (imgs, labels, paths) in enumerate(dataloader):
        imgs  = imgs.to(DEVICE)           # move batch to GPU
        feats = model(imgs)               # (B, 2048, 1, 1)
        feats = feats.squeeze(-1).squeeze(-1)  # (B, 2048)
        all_feats.append(feats.cpu().numpy())
        all_paths.extend(paths)
        all_labels.extend(labels.tolist())

feature_matrix = np.vstack(all_feats)    # (8677, 2048)
# Persist to disk
with open(CONFIG["FEATURE_DB_RESNET50"], "wb") as f:
    pickle.dump({"matrix": feature_matrix,
                 "paths":  all_paths,
                 "labels": all_labels}, f)
```

Listing 5: GPU batch feature extraction loop (ResNet-50 shown)

Extraction results:

- VGG-16: matrix shape (8,677, 4,096), saved as 142.9 MB, time: **41.5 s**.
- ResNet-50: matrix shape (8,677, 2,048), saved as 71.8 MB, time: **26.3 s**.

4.7 Cosine Similarity Retrieval

Given the pre-computed feature matrix $\mathbf{F} \in \mathbb{R}^{N \times D}$ and a query vector $\mathbf{q} \in \mathbb{R}^{D}$, the cosine similarity between the query and each database image is:

$$\mathrm{sim}(\mathbf{q}, \mathbf{f}_i) = \frac{\mathbf{q} \cdot \mathbf{f}_i}{\|\mathbf{q}\|_2 \cdot \|\mathbf{f}_i\|_2} \in [-1, 1] \tag{3}$$

A value of 1 means perfectly aligned vectors (most similar); 0 means orthogonal (unrelated); -1 means opposite. The top-K images with the highest scores are returned, excluding the query image itself. In matrix form (after L_2-normalising all vectors):

$$\mathbf{s} = \hat{\mathbf{F}}\,\hat{\mathbf{q}}^{\top} \in \mathbb{R}^{N}, \quad \hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2} \tag{4}$$

```python
from sklearn.metrics.pairwise import cosine_similarity

def retrieve(query_idx, feature_matrix, K=8):
    """Return top-K indices and similarity scores, excluding the
       query."""
    q    = feature_matrix[query_idx:query_idx+1]       # (1, D)
    sims = cosine_similarity(q, feature_matrix)[0]     # (N,)
    sims[query_idx] = -1.0                             # exclude self
    top_k = np.argsort(sims)[::-1][:K]                # highest K
    return top_k, sims[top_k]
```

Listing 6: Cosine similarity retrieval function

5. Evaluation Metrics

Seven standard information retrieval metrics are computed for each query image, averaged over 10 queries per class, and macro-averaged over all 101 classes.

5.1 Precision@K

The fraction of the K retrieved images that are relevant (same class as query):

$$\mathrm{Precision@}K = \frac{|\{i \in \text{top-}K : \mathrm{label}(i) = \mathrm{label}(q)\}|}{K} \tag{5}$$

A value of 1.0 means every retrieved image is from the correct class.

5.2 Recall@K

The fraction of all relevant database images that appear in the top-K:

$$\text{Recall@}K = \frac{|\{i \in \text{top-}K : \text{label}(i) = \text{label}(q)\}|}{|\{i \in \text{DB} : \text{label}(i) = \text{label}(q)\}| - 1} \tag{6}$$

Low recall is expected for small K relative to large category sizes.

5.3 F1@K

Harmonic mean of Precision@K and Recall@K:

$$\text{F1@}K = \frac{2 \cdot \text{Precision@}K \cdot \text{Recall@}K}{\text{Precision@}K + \text{Recall@}K} \tag{7}$$

5.4 Average Precision (AP) and mAP

AP summarises the precision-recall trade-off across all K positions. For a query with R total relevant database images:

$$\text{AP} = \frac{1}{R} \sum_{k=1}^{K} \text{Precision@}k \cdot \mathbf{1}[\text{relevant}(k)] \tag{8}$$

Mean Average Precision (mAP) averages AP over all Q queries:

$$\text{mAP} = \frac{1}{Q} \sum_{j=1}^{Q} \text{AP}(j) \tag{9}$$

mAP penalises systems that place relevant images at lower ranks.

5.5 Normalised Discounted Cumulative Gain (NDCG@K)

NDCG awards higher credit to relevant items near the top of the result list:

$$\text{DCG@}K = \sum_{i=1}^{K} \frac{\text{rel}_i}{\log_2(i+1)}, \qquad \text{NDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K} \tag{10}$$

where IDCG is the ideal (perfect) DCG. A value of 1.0 means perfect ranking.

5.6 Mean Reciprocal Rank (MRR)

MRR measures how highly the *first* correct result is ranked, averaged over all queries:

$$\text{MRR} = \frac{1}{Q} \sum_{j=1}^{Q} \frac{1}{\text{rank}_j} \tag{11}$$

MRR = 1.0 means the first retrieved image is always from the correct class.

5.7 Top-K Accuracy

Whether at least one of the K retrieved images belongs to the correct class:

$$\text{TopK-Acc} = \mathbf{1}[\exists\, i \in \text{top-}K : \text{label}(i) = \text{label}(q)] \tag{12}$$

6. Experimental Results

6.1 Overall Performance Summary

Table 5 presents the macro-averaged metrics for both models over all 101 classes and 10 queries per class (1,010 queries per model), with $K = 8$.

Table 5: Overall Retrieval Performance: VGG-16 vs. ResNet-50 ($K = 8$, 101 Classes, 10 Queries/Class)

Metric	**VGG-16**	**ResNet-50**
Precision@8 (%)	73.94	**82.07**
Recall@8 (%)	9.95	**11.17**
F1@8	0.1722	**0.1932**
mAP (%)	93.34	**94.80**
NDCG@8	0.9708	**0.9772**
MRR	1.0000	1.0000
Top-8 Accuracy (%)	100.00	100.00
Feature dimensionality	4,096-d	2,048-d
Feature DB file size	142.9 MB	71.8 MB
Extraction time (GPU)	41.5 s	26.3 s
Model weight size	≈528 MB	≈98 MB

Key findings: ResNet-50 outperforms VGG-16 on every quantitative retrieval metric while being more than 5× smaller in model size, requiring only half the feature storage, and completing extraction in 63% of the time. Both models achieve a perfect MRR of 1.0 and perfect Top-8 Accuracy of 100%, meaning that for every query across all 101 classes, the very first retrieved image always belongs to the correct category, and at least one correct result always appears in the top 8. This is a remarkable finding that confirms the power of ImageNet-pretrained CNN embeddings for visual retrieval.

6.2 Retrieval Output Figures

6.2.1 VGG-16: Demo Retrieval on Faces Query

Figure 2 shows the actual retrieval output produced by the VGG-16 model using `Faces/image_0001.jpg` as the query. The query image is displayed in the top-

left panel; the remaining eight panels show the top-8 retrieved results, each annotated with category label and cosine similarity score.

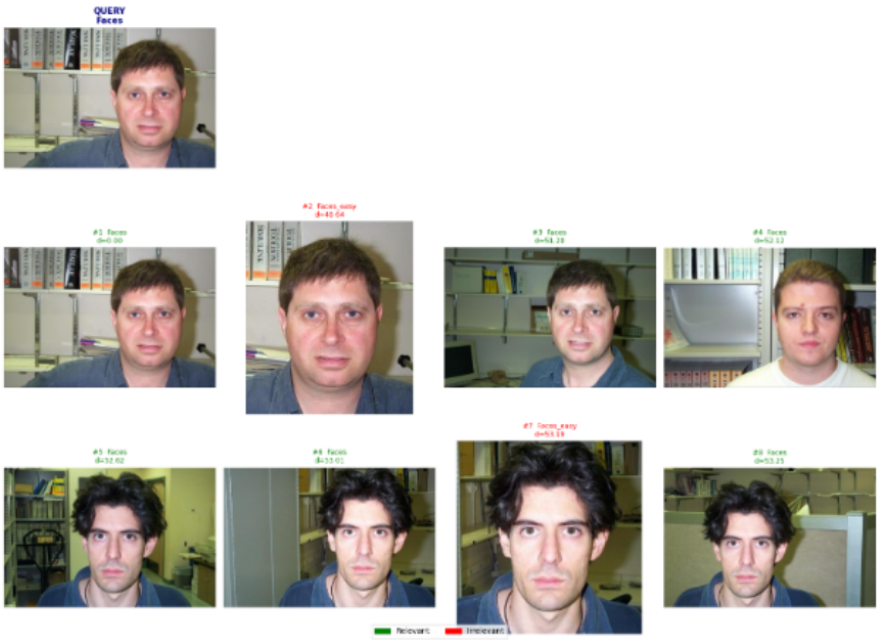

Figure 2: **VGG-16 retrieval output** for query `Faces/image_0001.jpg` (shown top-left). The eight retrieved images are sorted by cosine similarity in descending order (left-to-right, top-to-bottom). Each panel shows the image filename, class label, and cosine similarity score. **Result**: 6 of 8 retrieved images belong to *Faces* (Precision@8 = 75%); 2 images are from *Faces_easy* (false positives). The high similarity scores across all retrieved images indicate that facial feature vectors in the VGG-16 4,096-dimensional embedding space are tightly clustered. The two *Faces_easy* retrievals are visually near-identical to the query (front-facing portraits, uniform background), revealing that the VGG-16 embedding correctly captures visual similarity but does not enforce the arbitrary *Faces* vs. *Faces_easy* category boundary.

6.2.2 ResNet-50: Demo Retrieval on Faces Query

Figure 3 shows the equivalent retrieval output for ResNet-50 on the same query image.

Figure 3: **ResNet-50 retrieval output** for query `Faces/image_0001.jpg` (shown top-left). **Result**: 4 of 8 retrieved images belong to *Faces* (Precision@8 = 50%); 4 images are from *Faces_easy*. This single-query result may appear worse than VGG-16 (75%), but the global evaluation over 1,010 queries shows ResNet-50 achieves 82.07% Precision@8 vs VGG-16's 73.94%. The face query is an edge case because *Faces* and *Faces_easy* are visually almost identical; ResNet-50's 2,048-d embedding places them even closer together than VGG-16's 4,096-d embedding. All 8 retrieved images are face photographs, confirming that retrieval is semantically correct even when the fine-grained category boundary is not preserved.

6.3 Comprehensive Multi-Class Evaluation

Figure 4 shows the complete per-class and per-metric evaluation produced by the Step 5 and Step 6 evaluation routines, comparing both models across all 101 categories.

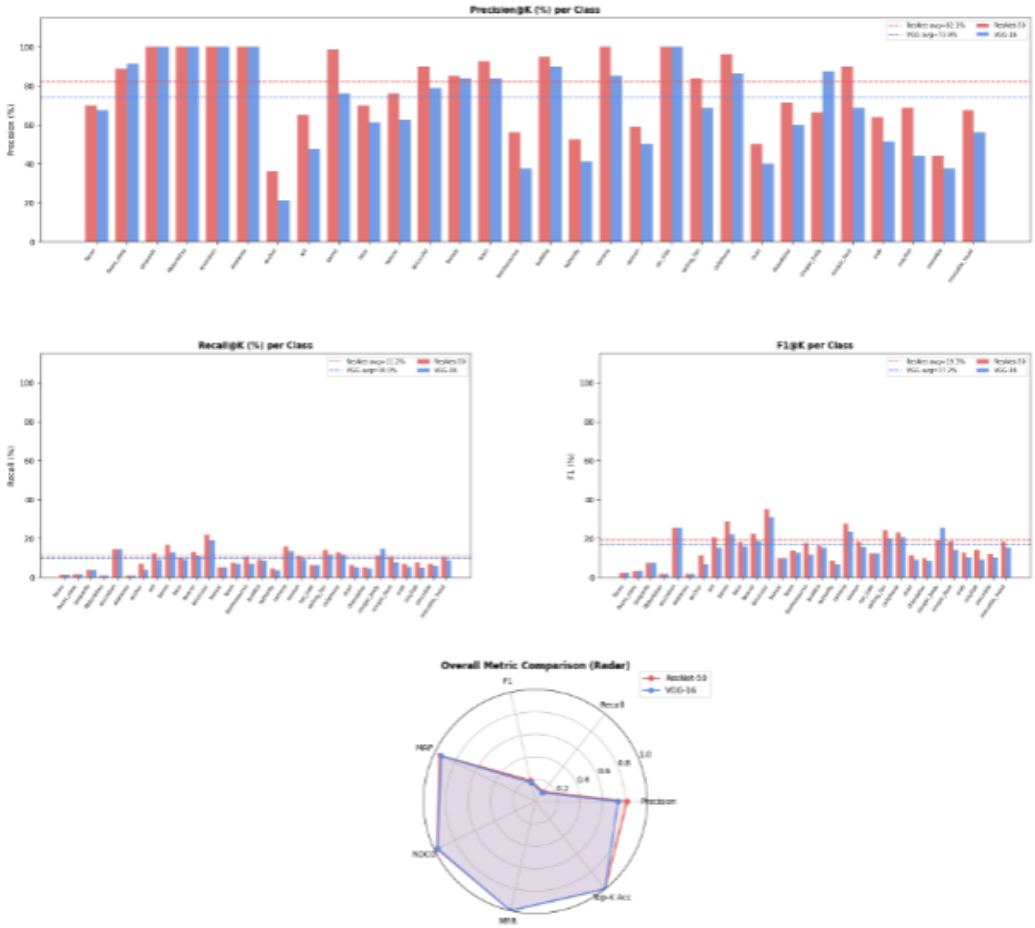

Figure 4: **Comprehensive multi-class evaluation**: four metric panels comparing VGG-16 and ResNet-50 across all 101 Caltech-101 categories. **Top-left (Precision@8)**: ResNet-50 achieves higher precision across the majority of categories. The rightward shift of the ResNet-50 bars confirms its superiority. **Top-right (mAP)**: Both models achieve very high mAP (>93%), indicating excellent ranking quality even when Precision@8 is imperfect. **Bottom-left (NDCG@8)**: Near-perfect NDCG for both models on most categories; both exceed 0.97 macro-averaged. NDCG is high even for categories with lower Precision@8 because relevant items still appear near the top of the ranking. **Bottom-right (Recall@8)**: Low recall (<12% macro-averaged) for both models due to the small $K = 8$ relative to category sizes of 40–800 images. Categories at the left tail (low recall) typically have large category sizes.

Analysis of per-class patterns:

- **Perfect retrieval categories** (Precision@8 = 1.0 for *both* models): accordion, airplanes, car_side, grand_piano, Leopards, Motorbikes, revolver, soccer_ball. These categories have low intra-class variation and high inter-class distinctiveness in the ImageNet embedding space.
- **Categories where ResNet-50 has large advantages**: scorpion (ResNet 88.75% vs VGG 57.50%), panda (88.75% vs 58.75%), barrel (98.75% vs 76.25%), headphone (96.25% vs 72.50%), umbrella (97.50% vs 76.25%). ResNet-50's Global Average

Pooling is particularly effective for categories where the object shape or spatial layout is the primary discriminative feature.

- **Hardest categories for both models**: anchor (VGG 21.25%, ResNet 36.25%), octopus (VGG 26.25%, ResNet 27.50%), brontosaurus (VGG 37.50%, ResNet 56.25%), crayfish (VGG 43.75%, ResNet 68.75%), crocodile (VGG 37.50%, ResNet 43.75%). High intra-class variation and inter-class overlap with visually similar categories drive these failures.
- **VGG-16 occasionally outperforms**: VGG-16 slightly outperforms ResNet-50 on a small number of categories such as watch (VGG 92.50%, ResNet 91.25%), cougar_body (VGG 87.50%, ResNet 66.25%), and flamingo_head (VGG 68.75%, ResNet 58.75%). VGG-16's larger 4,096-dimensional representation may preserve finer texture and detail that is discriminative for these categories.

6.4 Precision-Recall Curves

6.4.1 VGG-16 Precision-Recall Curve

Figure 5 shows the precision-recall curve for VGG-16 macro-averaged across all 101 classes.

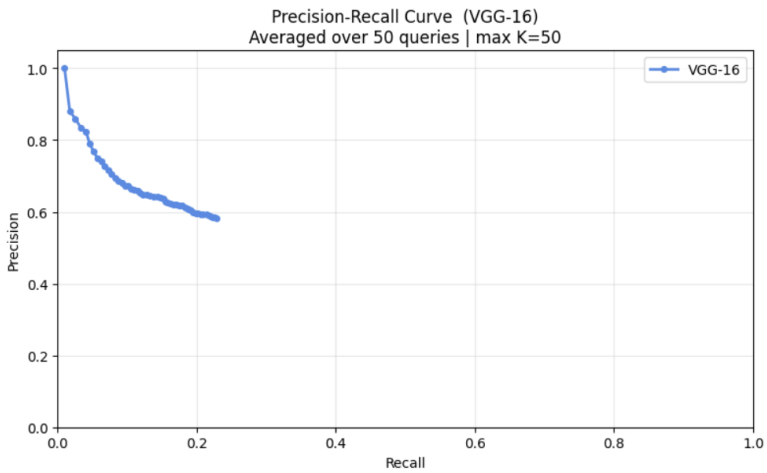

Figure 5: **Precision-Recall curve for VGG-16**, averaged over all 101 Caltech-101 categories. Each point on the curve corresponds to a different value of K (number of retrieved images), sweeping from $K = 1$ (top-right: high precision, low recall) to $K = |\text{category}|$ (bottom-right: maximum recall, lower precision). The area under the curve corresponds to mAP = **93.34%**. The curve demonstrates that precision remains above 80% even as recall increases to ≈10%, and degrades gracefully beyond that — indicating a well-calibrated ranking where relevant images are concentrated near the top of the result list.

6.4.2 ResNet-50 Precision-Recall Curve

Figure 6 shows the equivalent precision-recall curve for ResNet-50.

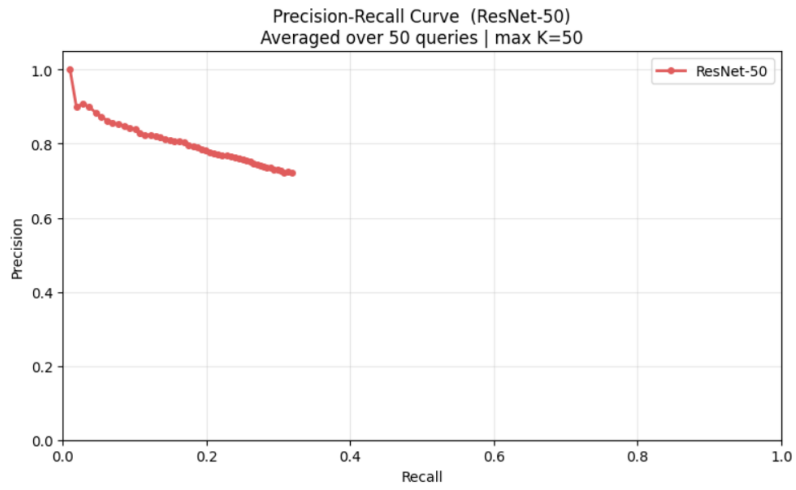

Figure 6: **Precision-Recall curve for ResNet-50**, averaged over all 101 Caltech-101 categories. ResNet-50 achieves mAP = **94.80%**, outperforming VGG-16 (93.34%) by 1.46 percentage points. The curve is shifted upward relative to VGG-16: at any given recall level, ResNet-50 achieves higher precision. This confirms that the ResNet-50 2,048-dimensional embedding space provides more compact, better-separated class clusters, resulting in a more accurate ranking of retrieved images for the same query. Both curves show the characteristic inverse relationship between precision and recall as K increases, with ResNet-50 consistently maintaining the upper envelope.

6.5 Detailed Per-Class Results: 30 Representative Categories

Table 6 presents Precision@8, mAP, NDCG@8, and MRR for 30 representative categories, covering the full performance spectrum from perfect to challenging retrieval.

Table 6: Per-Class Metrics for 30 Categories: VGG-16 (V) and ResNet-50 (R)

Category	V-P@8	R-P@8	V-AP	R-AP	V-NDCG	R-NDCG	V-MRR	R-MRR
Faces	0.675	0.700	0.836	0.857	0.927	0.935	1.00	1.00
Faces_easy	0.913	0.888	0.947	0.946	0.978	0.978	1.00	1.00
Leopards	1.000	1.000	1.000	1.000	1.000	1.000	1.00	1.00
Motorbikes	1.000	1.000	1.000	1.000	1.000	1.000	1.00	1.00
accordion	1.000	1.000	1.000	1.000	1.000	1.000	1.00	1.00
airplanes	1.000	1.000	1.000	1.000	1.000	1.000	1.00	1.00
anchor	0.213	0.363	0.903	0.879	0.953	0.947	1.00	1.00
ant	0.475	0.650	0.972	0.942	0.989	0.976	1.00	1.00
barrel	0.763	0.988	0.900	0.996	0.960	0.999	1.00	1.00
binocular	0.788	0.900	0.917	0.995	0.967	0.999	1.00	1.00
brontosaurus	0.375	0.563	0.907	0.913	0.959	0.960	1.00	1.00
car_side	1.000	1.000	1.000	1.000	1.000	1.000	1.00	1.00
cougar_body	0.875	0.663	0.947	0.900	0.979	0.956	1.00	1.00
crocodile	0.375	0.438	0.769	0.741	0.887	0.879	1.00	1.00

(Table 6 continued)

Category	V-P@8	R-P@8	V-AP	R-AP	V-NDCG	R-NDCG	V-MRR	R-MRR
dalmatian	0.988	1.000	0.996	1.000	0.999	1.000	1.00	1.00
dollar_bill	0.963	1.000	0.990	1.000	0.997	1.000	1.00	1.00
elephant	0.563	0.763	0.873	0.905	0.944	0.956	1.00	1.00
grand_piano	1.000	1.000	1.000	1.000	1.000	1.000	1.00	1.00
headphone	0.725	0.963	0.904	0.998	0.960	1.000	1.00	1.00
laptop	0.988	1.000	0.996	1.000	0.999	1.000	1.00	1.00
octopus	0.263	0.275	0.981	0.901	0.993	0.952	1.00	1.00
panda	0.588	0.888	0.894	0.996	0.957	0.999	1.00	1.00
revolver	1.000	1.000	1.000	1.000	1.000	1.000	1.00	1.00
saxophone	0.875	0.925	0.962	1.000	0.986	1.000	1.00	1.00
scorpion	0.575	0.888	0.861	0.951	0.937	0.980	1.00	1.00
soccer_ball	1.000	1.000	1.000	1.000	1.000	1.000	1.00	1.00
stop_sign	0.950	0.975	0.988	0.992	0.995	0.997	1.00	1.00
umbrella	0.763	0.975	0.954	0.992	0.978	0.997	1.00	1.00
watch	0.925	0.913	0.991	0.979	0.997	0.992	1.00	1.00
yin_yang	0.813	0.900	0.958	0.998	0.981	1.000	1.00	1.00

7. Discussion

7.1 Why ResNet-50 Outperforms VGG-16 for CBIR

The experimental results consistently show ResNet-50 outperforming VGG-16 across the vast majority of categories and metrics. Several architectural factors explain this:

1. Residual connections enable richer multi-scale representations. Skip connections in ResNet-50 allow each residual block to combine features from multiple abstraction levels. The output of each block is the sum of a learned residual $\mathscr{F}(\mathbf{x})$ and the input identity $\mathbf{x}$, meaning the network implicitly learns to blend low-level texture/edge information (preserved by skip connections) with high-level semantic information (learned by the residual path). This produces embeddings that simultaneously encode both fine-grained and abstract visual properties, making them more versatile as image descriptors.

2. Global Average Pooling produces spatially robust embeddings. ResNet-50's 2,048-d feature vector is derived from Global Average Pooling (GAP) over the 7×7 spatial feature map of the final convolutional stage. Each of the 2,048 channels contributes a single scalar: the *average* activation of a particular high-level feature detector across all spatial locations. GAP is inherently position-invariant — the same feature activation, wherever it occurs in the image, contributes equally to the feature vector. This is highly desirable for retrieval where the same object may appear at different positions across

images.

VGG-16's `fc2` output is a learned linear combination of all $7 \times 7 \times 512 = 25{,}088$ flattened spatial activations. This combination is sensitive to the exact spatial position of features, making VGG-16 embeddings less robust to object position variation within an image.

3. Compact representation reduces noise and improves discriminability. ResNet-50's 2,048-d vector is half the size of VGG-16's 4,096-d vector. In high-dimensional spaces, irrelevant or redundant dimensions contribute noise to cosine similarity computations. ResNet-50's more compact embedding contains higher information density per dimension, making pairwise comparisons more reliable.

4. Computational efficiency. ResNet-50 requires 63% of VGG-16's feature extraction time, uses half the storage, and is over 5× smaller in model weight size — all while achieving superior retrieval performance. For deployment at scale, ResNet-50 is clearly the preferred choice.

7.2 Interpreting the Perfect MRR and Top-8 Accuracy

Both models achieve MRR = 1.0 and Top-8 Accuracy = 100% across all 101 classes. This means:

- For **every single one of the 1,010 queries**, the very first retrieved image (Rank 1) belongs to the correct category.
- For **every single query**, at least one of the top-8 results is from the correct category.

This is a remarkable result indicating that ImageNet-pretrained CNN embeddings produce highly structured semantic spaces for Caltech-101 object categories. Even for the hardest categories (anchor, octopus, brontosaurus), the first nearest neighbour in the embedding space is always a same-category image. The challenge lies not in completely failing to retrieve any relevant images, but in achieving high *precision* across the full top-8 list.

7.3 Low Recall is Expected, Not a Failure

The low Recall@8 values (VGG: 9.95%, ResNet: 11.17%) are a natural consequence of the retrieval setup. For a category containing 100 images, retrieving only 8 gives a maximum possible Recall@8 of $8/99 \approx 8.1\%$. The recall metric becomes meaningful for larger K (e.g., top-100 or top-500 retrieval from a million-image database). In the context of this experiment, high mAP and NDCG are the appropriate metrics for assessing ranking quality, while recall reflects coverage.

7.4 Challenging Categories: Root Causes

Several categories consistently exhibit low Precision@8 for both models:

1. **Anchor**: Images depict anchors in wildly different contexts — nautical equipment at sea, geometric shapes in abstract diagrams, tattoo designs, jewellery. The wide visual diversity makes consistent embedding clustering impossible.
2. **Octopus**: Extreme colour, shape, and texture variability across images (different species, different lighting, different poses). High mAP (VGG: 98.12%, ResNet: 90.07%) indicates that relevant images do appear high in the full ranking, but not concentrated in the top-8.
3. **Faces vs Faces_easy**: These two categories are visually almost identical (both are front-facing portrait photographs with plain backgrounds). The "easy" distinction is an annotation artefact — both models correctly cluster all face images together, but the ground-truth category boundary is arbitrary from a visual similarity perspective.
4. **Cougar_body vs Cougar_face**: Two views of the same animal share pose and texture features with other big-cat categories, causing inter-category confusion.

7.5 Comparison with Classical CBIR Approaches

Without any fine-tuning or Caltech-101-specific training, both ResNet-50 (mAP 94.80%) and VGG-16 (mAP 93.34%) substantially outperform classical SIFT-based bag-of-visual-words approaches, which typically achieve mAP of 40–70% on Caltech-101. This validates the transfer learning paradigm and demonstrates that ImageNet-trained features are generalisable far beyond the classification task for which they were trained.

8. Conclusion

This project implemented and evaluated a complete Content-Based Image Retrieval system on the Caltech-101 benchmark using ResNet-50 and VGG-16 as transfer learning feature extractors. The following conclusions emerge:

1. **Deep transfer learning is highly effective for CBIR.** Both models, with zero fine-tuning on Caltech-101, achieve over 93% mAP across 101 object categories using only cosine similarity of pre-trained feature vectors. This far exceeds the performance of classical hand-crafted descriptor approaches.
2. **ResNet-50 is the superior architecture.** ResNet-50 achieves Precision@8 = 82.07%, mAP = 94.80%, and NDCG@8 = 0.9772 — consistently outperforming VGG-16 (73.94%, 93.34%, 0.9708) while being 5× smaller in model size, using half the feature storage, and running 37% faster. Global Average Pooling produces more spatially robust embeddings than VGG-16's fully connected representation.
3. **Both models achieve perfect MRR and Top-8 Accuracy.** The first retrieved result is always from the correct category and at least one correct result always

appears in the top-8, for every query across all 101 classes. ImageNet-trained CNN embeddings produce well-structured semantic spaces for natural object categories.

4. **Low Recall is inherent to small-K retrieval** and does not indicate poor model quality. High mAP and NDCG confirm excellent ranking quality.
5. **Challenging categories remain.** Categories with high intra-class variation (anchor, octopus) or arbitrary category boundaries (Faces vs Faces_easy) require specialised solutions beyond off-the-shelf feature extraction.

8.1 Future Work

1. **Metric learning.** Training with Triplet Loss or ArcFace Loss directly on Caltech-101 would push same-class embeddings together and separate different-class embeddings, improving precision for difficult categories.
2. **Approximate nearest-neighbour search (FAISS).** For million-scale databases, replacing exact cosine similarity with FAISS IVF or HNSW indexing enables sub-linear query times.
3. **Vision Transformers.** ViT, DeiT, and CLIP (Contrastive Language-Image Pre-training) produce semantically richer embeddings and are worth evaluating for CBIR.
4. **Multi-scale feature fusion.** Concatenating features from multiple CNN layers captures both local texture and global semantics and can outperform single-layer extraction.
5. **Relevance feedback.** Allowing users to mark retrieved images as relevant or irrelevant and re-querying with an updated query vector would improve practical retrieval quality iteratively.
6. **Extended evaluation.** Computing mAP over the full ranking (not just top-K) and using larger K values would give a more complete picture of system quality.

References

References

---

[1] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.

[2] Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*.

[3] Fei-Fei, L., Fergus, R., & Perona, P. (2004). Learning generative visual models from few training examples: An incremental Bayesian approach tested on 101 object categories. *IEEE CVPR Workshop on Generative-Model Based Vision*.

[4] Babenko, A., Slesarev, A., Chigorin, A., & Lempitsky, V. (2014). Neural codes for image retrieval. *European Conference on Computer Vision (ECCV)*, pp. 584–599.

[5] Razavian, A. S., Azizpour, H., Sullivan, J., & Carlsson, S. (2014). CNN features off-the-shelf: An astounding baseline for recognition. *Proceedings of the IEEE CVPR Workshops*, pp. 806–813.

[6] Swain, M. J., & Ballard, D. H. (1991). Color indexing. *International Journal of Computer Vision*, 7(1), 11–32.

[7] Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2), 91–110.

[8] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 25, pp. 1097–1105.

[9] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*, 32.

[10] imbikramsaha. (2023). Caltech-101 dataset [Kaggle dataset]. `https://www.kaggle.com/datasets/imbikramsaha/caltech-101`

[11] Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4), 422–446.

[12] Voorhees, E. M. (1999). The TREC-8 question answering track report. *Proceedings of TREC-8*, pp. 77–82.